Modern **A**rduino **D**evelopment **E**nvironment for **DevOps**

https://github.com/DaFessor/made4devops

Users Guide

Introduction *3*

1 Installing the necessary software *3*

1.1 Usbipd 4

2 Work with a new Made4DevOps project *4*

3 Doing a development session *4*

3.1 Attach an Arduino 4

3.1.1 Step 1 – Make sure WSL is configured correctly 4

3.1.2 Step 2 - Attach the Arduino to an USB port 5

3.1.3 Step 3 - Make the Arduino available in WSL 5

3.1.3.1 Step 3A – bind the Arduino with usbipd (only if first time use of a specific Arduino) 5

3.1.3.2 Step 3B – attach the Arduino to a client (WSL) (only once at the start of a development session) 6

3.2 Open project in dev container 6

3.3 Open workspace file 7

3.4 Use the task menu 9

3.5 Make changes to project 9

3.5.1 Add new target firmware files 9

3.5.2 General notes on creating tests for target and native environments 9

3.5.3 Add new target test files 10

3.5.4 Add new native/host files 10

3.6 Create and run new firmware and tests on Arduino 11

3.7 Create and run new native/host tests 11

3.8 Push updated project to GitHub 12

3.9 View info on latest version on GitHub 12

3.10 Stop a session 13

4 Removing unnecessary software *13*

5 Extra features *13*

5.1 Sep4 hardware library 13

5.2 Sep4 HW test 13

6 How do I *13*

6.1 - get access to the raw project folder? 13

6.2 - reconnect to an Arduino that's already running my software? 14

7 Work flow overview *15*

Figures

Figure 1 - General flow....3
Figure 2 - Opening the workspace....7
Figure 3 - Choosing the workspace file....7
Figure 4 - The virtual subprojects in the workspace....8
Figure 5 - The task menu....9
Figure 6 - Viewing test coverage report....12
Figure 7 - Workflow Overview....15

Introduction

Made4DevOps (https://github.com/DaFessor/made4devops) is a framework that allows developers to create software for the Arduino ATMega2560 in an environment that fully supports DevOps principles, especially CI (Continuous Integration) and to some extent CD (Continuous Deployment). It does so by taking advantage of container technology, running all the development tools inside a development container. Since the intention with Made4DevOps was to embrace and support DevOps principles, all parts of the development lifecycle, including flashing firmware to the Arduino, is supported.

This document provides general information about Made4DevOps, instructions for installing the software needed to work with Made4DevOps and some help with respect to how one works with Made4DevOps. It is still a work in progress, though, so do not hesitate to ask the instructor for help.

Figure 1 - General flow on page 3 provides an overview of the workflow associated with Made4DevOps.

General flow	***Info in section-page***
Start using Made4DevOps – install the necessary software	1 p3
Work with a (new) project – clone or fork Made4Devops	2 p4
Doing a development session	3 p4
Bind a new Arduino and/or attach a known Arduino	3.1 p4
Open project in dev container	3.2 p6
Open workspace file	3.3 p7
Work with project, make changes	3.5 p9
Add new target firmware .c and .h files	3.5.1 p9
Add new target test .c and .h files	3.5.3 p10
Add new native/host test .c and .h files	3.5.4 p10
Create and run new firmware or tests on the Arduino	3.6 p11
Create and run new native/host tests on laptop	3.7 p11
Push updated project to GitHub	3.8 p11
View info on latest version on Github	3.9 p12
Stop a session	3.10 p13
Stop using Made4DevOps – uninstall unnecessary software	4 p13

FIGURE 1 - GENERAL FLOW

1 Installing the necessary software

You will need this software:

a. Windows Subsystem for Linux v2 (WSL2)

b. Docker Desktop
c. VSCode
d. Usbipd

You should already have a-c installed. Note: *VSCode must have the extension called "Dev Containers" installed*.

1.1 Usbipd

The new thing here is usbipd. Usbip is project that aims to provide access to usb devices over the ip protocol (see https://usbip.sourceforge.net/ for more information). Usbipd is a Windows implementation of usbip. It runs as a background service once it is installled, and we need it to be able to access usb devices (i.e. and Arduino) from inside WSL and containers.

To install it:

1) Go to https://github.com/dorssel/usbipd-win/tags and find the latest version (5.3.0 as of writing).
2) Click on "Downloads"
3) Choose the msi-file (e.g. `usbipd-win_5.3.0_x64.msi`) and download it.
4) Simply install usbipd by clicking on the msi-file.

2 Work with a new Made4DevOps project

To work with a new Made4DevOps project, you must either:

- **Clone it.**
  That's what we usually do when working with premade Github projects in the courses. That gives you a local copy on your PC. That works fine if you only want to play with it, but you will not be able to push changes back to the Github repository.

- **Fork it.**
  This is what you must do if you want to get your own personal copy *on Github*, that allows you to push changes back.
  Example: if you want to use Made4DevOps in your semester project you will need to make your own fork of Made4Devops.

In any, before moving on you must clone the appropriate Github repository to your PC.

3 Doing a development session

Before starting a development session, it's a good idea to first attach any Arduino you want to work with. You *can* do it later, but it obviously must be done before you can flash anything to the Arduino

3.1 Attach an Arduino

Attaching an Arduino is a three step process:

3.1.1 Step 1 – Make sure WSL is configured correctly

Open Ubuntu in a shell/terminal. Execute these commands (without the // comments):

```
> cd /etc              // Change to the /etc folder
> cat wsl.conf         // Show the contents of the file wsl.conf
[boot]

systemd=true
```

```
[user]
default=stek
```

If the contents you see look like this (the user name will be different), and the [boot]-thing is there, you are done and can move to step 2.

Otherwise, If there's no [boot] part you need to edit the /etc/wsl.conf file and add these two lines. Use micro like this

```
> micro wsl.conf
```

Or VSCode like this

```
> code wsl.conf
```

Add the two lines to the file:

```
[boot]
systemd=true
```

Once you have updated the /etc/wsl.conf file, you need to restart wsl. Follow this procedure:

1. Close all Ubuntu/WSL shells/terminals
2. Type this in a Powershell terminal:
   wsl.exe –shutdown
3. Start a new Ubuntu/WSL shell/terminal to make sure that WSL starts again.

Now you can move to step 2.

3.1.2 Step 2- Attach the Arduino to an USB port

That's just business as usual.

3.1.3 Step 3 - Make the Arduino available in WSL

To make Arduino available in WSL, follow this procedure. If this is the first time you want to use that particular Arduino, step 3A is necessary. Next time you connect the same Arduino you can just skip step 3A and go straight to step 3B

3.1.3.1 Step 3A – bind the Arduino with usbipd (only if first time use of a specific Arduino)

Make sure the Arduino is plugged in and then start a Powershell with Administrator rights. Type this command:

```
PS C:\Users\stek> usbipd.exe list
```

This makes usbipd list the usb devices it sees and knows about:

```
Connected:
BUSID  VID:PID    DEVICE                    STATE
2-4    2341:0042  Seriel USB-enhed (COM4)   Not shared
2-7    30c9:000f  HP HD Camera              Not shared
2-8    06cb:00f0  Synaptics FS7605 Touch    Not shared
2-14   8087:0026  Wireless Bluetooth(R)     Not shared
Persisted:
GUID                                   DEVICE
```

Busid 2-4

We can see the Arduino has been attached to COM4 and that this particular Arduino is in state *Not shared*. We want to make it available inside WSL, so we will make a note of the bus id (2-4 in the example above. To make it available we need to ***bind*** in usbipd it first. Binding means that usbipd will automatically recognize the Arduino in the future and make it sharable. So, we type:

```
PS C:\Users\stek> usbipd.exe bind -b 2-4
// Wait 10 seconds before typing the next command
PS C:\Users\stek> usbipd.exe list
```

The first command (usbipd.exe bind -b 2-4) binds the Arduino and the second command (usbipd.exe list) shows the result of binding the device (notice that the Arduino has gone from "Not shared" to "Shared":

```
Connected:
BUSID  VID:PID    DEVICE                    STATE
2-4    2341:0042  Seriel USB-enhed (COM4)   Shared
2-7    30c9:000f  HP HD Camera              Not shared
2-8    06cb:00f0  Synaptics FS7605 Touch    Not shared
2-14   8087:0026                            Not shared
Persisted:
GUID                                   DEVICE
```

Shared

This binding step is only necessary the first time you want to use a specific Arduino.

3.1.3.2 Step 3B – attach the Arduino to a client (WSL) (only once at the start of a development session)

The Arduino is now in state Shared which means we can use it over IP. To make the Arduino available inside WSL, the final step is to ***attach*** it – that will connect the Arduino (COM4) to a client. Your PC may choose a different bus-id every time you physically plug your Arduino in, so use the `usbipd.exe list` command first to check the status/bus-id after plugging your Arduino in, before you attach.

Once you have determined the bus-id for your Arduino, we again start a Powershell for this, but *you don't have to run as administrator for this step*:

In this case we want to attach it to WSL:

```
PS C:\Users\stek> usbipd.exe attach -a -w -b 2-4
```

The -a stands for auto attach and -w means attach to WSL. Auto attach means that usbipd will automatically re-attach the Arduino even if gets unplugged and then plugged in again, and that's what we want. The command will keep on running until we either

- press Ctrl-C,
- close the Powershell terminal or
- shut down the PC

In general, you should just minimize the Powershell window and let it run until your Made4DevOps session is done.

3.2 Open project in dev container

Open the folder with your local copy of Made4DevOps with VSCode. At some point VSCode will ask you if you want to reopen the project in a container, which you should then do (just click the button).

3.3 Open workspace file

When you open the project in a newly generated devcontainer, you should open the workspace file like this:

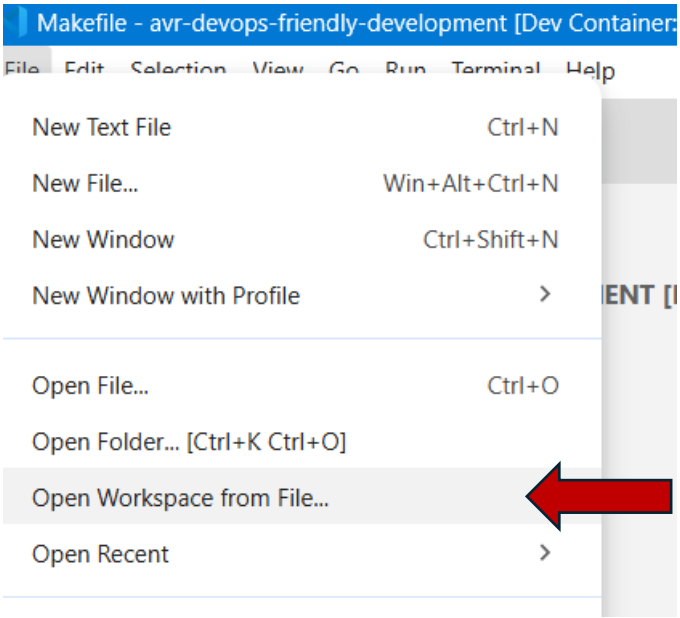


FIGURE 2 - OPENING THE WORKSPACE

Open the file called .code-workspace:

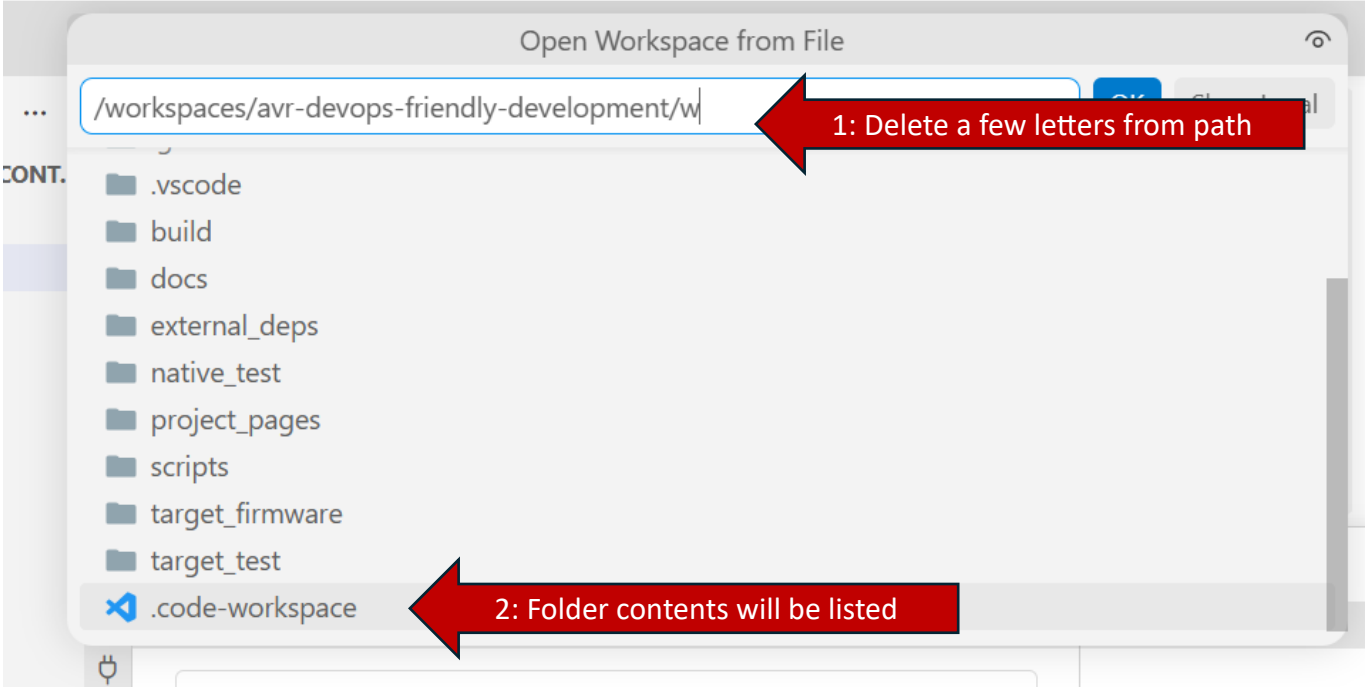


FIGURE 3 - CHOOSING THE WORKSPACE FILE

You only need to do this when the container is entirely new, next time you use the container you will start up in the workspace. The workspace file provides a nice, virtual project with 6 different, smaller subprojects shown below in Figure 4. It also hides all the nitty-gritty details and hides files that you should (probably) not change, allowing you to focus on the source code files for you project.

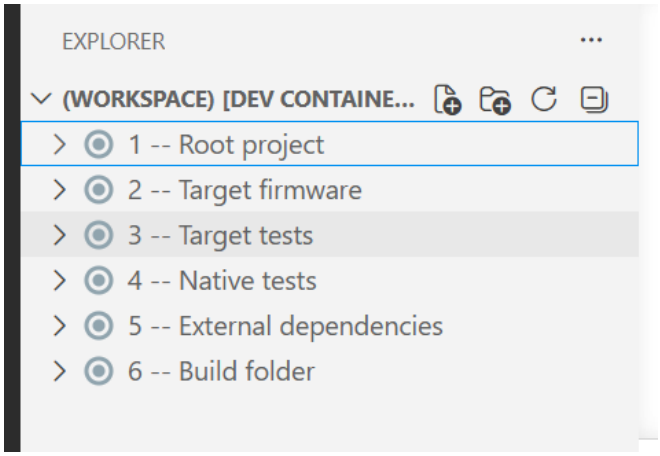


FIGURE 4 - THE VIRTUAL SUBPROJECTS IN THE WORKSPACE

These are the six subprojects:

- **Root project** : this is just a pseudo project that contains the root file system of the project with all the stuff that's not inside the other subprojects. You should rarely need to make any changes here.
- **Target firmware** : This is where all your "production code" for the Arduino is placed. Update, change, add and remove files in the src/ folder as you please.
- **Target tests** : This is where all test code you want to run on the Arduino is placed. Update, change, add and remove files in the src/ folder as you please.
- **Native tests** : This is where all test code you want to run on your laptop is placed. Update, change, add and remove files in the src/ folder as you please.
- **External dependencies** : This is where external dependencies will automatically be downloaded and placed whenever they are needed. By downloading deps when they are needed, rather than simply putting them directly inside the project, the project footprint in the repository is kept as small as possible. Don't make changes here unless you want to add or remove external dependencies.
- **Build folder** : *ALL* generated files are placed here. Compiler out output, hex files, log files whatever, goes into this subproject/folder. You can browse anything here, but there's no need to delete or clean up anything that's handled by the relevant tasks (see below).

3.4 Use the task menu

All operations (build, flash, run code etc.) is handled via a task menu:

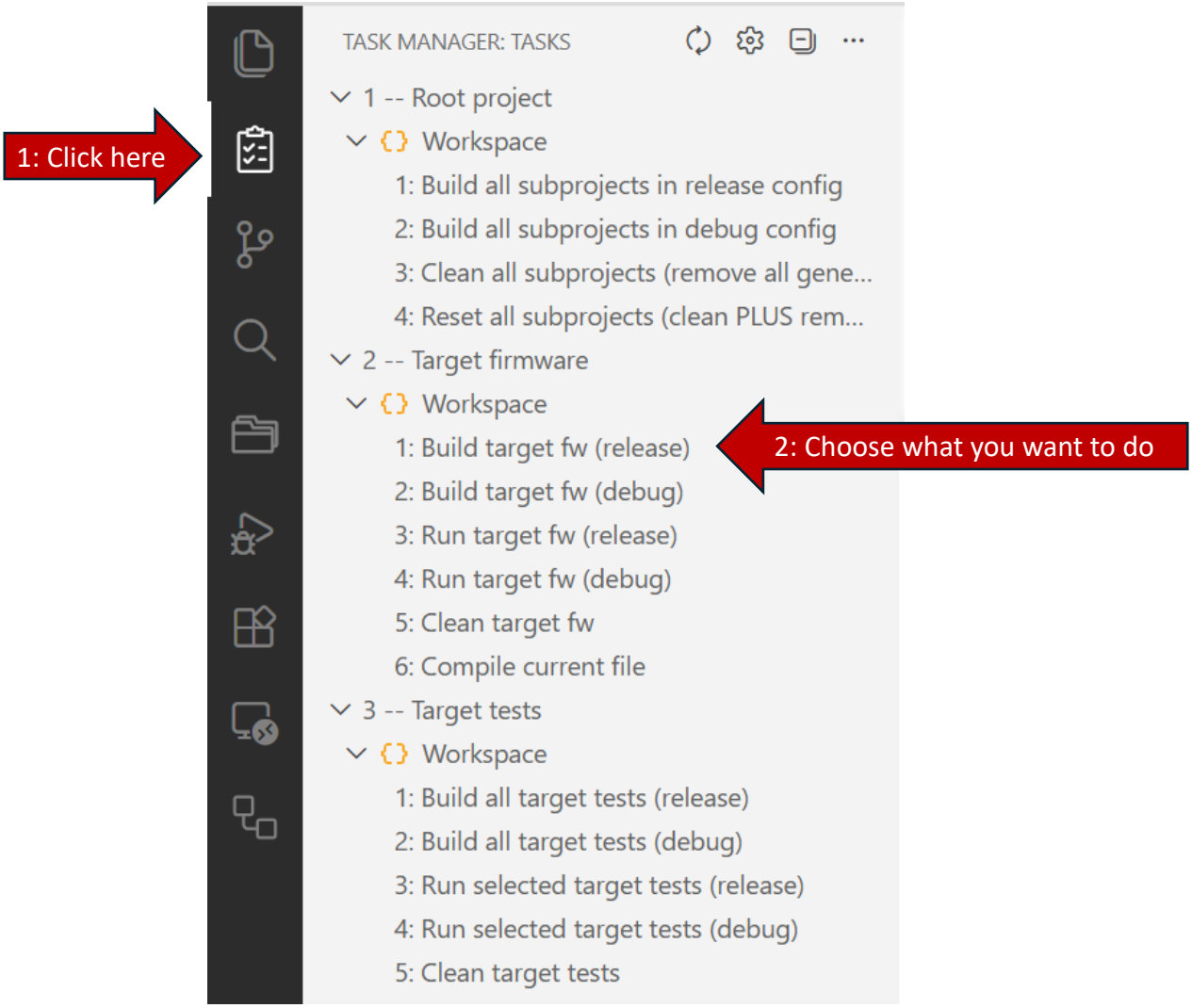


FIGURE 5 - THE TASK MENU

3.5 Make changes to project

At this point you are up and running, and can start to make changes to the project. Use the explorer to choose which subproject (target firmware, target tests etc.)

3.5.1 Add new target firmware files

Simply put all your Arduino product code files (*.c *.h) in the src/ folder. All code in the src/ folder will be picked up, compiled and placed in a flashable .hex file.

3.5.2 General notes on creating tests for target and native environments

On of the most powerful tools, if not the most powerful, is the use of fakes to isolate code you want to test by replacing all dependencies with fakes.

But it is not possible to use both a fake and a real version of a function in the *same* executable/flashable file. In the final stage of building an executable or flashable file a tool called a ***linker*** is used to merge all the .o files created by the compiler into a single runnable file. The .o files contain machine code plus some extra information about the symbols inside each .o file (symbols are basically just the named stuff

from your source code, i.e. names of functions and variables). A .o file may *reference* and/or *define* a symbol. The linker is responsible for making it possible for a function in, say, xxxx.c call (reference) a function defined in yyyy.c. The linker has two strict requirements:

- All symbols that are referenced to but not defined in an .o file MUST be defined in another .o file that is part of the build.
- There can only be 1 definition of any symbol.

Which all makes sense. The gist of this is that we often are often foreced to chop up our tests into several smaller executables/flashables to avoid mixing a fake and the real version of a function inside the same executable/flashable file.

To help with that Made4DevOps places a file called `test_list.mk` inside each of the target and native test folders. In the test_list.mk file you can define which tests executables you want to create and which specific .c source files should be made a part of each test executable/flashable file. The general format is rather simple (although the syntax is slightly odd):

$(eval $(call TEST_DEFINITION, test_mul_op.c, add_op.c sub_op.c))

Test file "Production" files

The red stuff is just (macro) boiler plate. The first "blue file" contains test code, and all the others (after the comma) are production files that should be used in the test. So, in this case `test_mul_op.c` should be placed in the test folder (target_test or native_test) and all the others are automatically picked up from the target_firmware folder.

3.5.3 Add new target test files

Simply put all your Arduino test code files (*.c *.h) in the src/ folder. All code in the src/ folder will be picked up, compiled and placed in one or more flashable .hex files. There are two test frameworks available, fff and unity.

Once you have added any .c files, you must update the file called `test_list.mk`. Look at the existing examples to see how that is done. You can choose to put all your tests in a single, flashable file or create multiple flashable files with different test suites, that's up to you you (it also depends on your use of fakes as described in section 3.5.2).

But that's the procedure for creating new target tests:

1) Add the necessary .c and .h files in the src/ folder
2) Update the `test_list.mk` to specify how you want your tests structured across one or more flashable files.

3.5.4 Add new native/host files

Simply put all your native/test code files (*.c *.h) in the src/ folder. All code in the src/ folder will be picked up, compiled and placed in a flashable .hex file. There are three test frameworks available, fff, unity and cmock.

Once you have added any any .c files, you must update the file called `test_list.mk`. Look at the existing examples to see how that is done. You can choose to put all your tests in a single, flashable file or create multiple flashable files with different test suites, that's up to you (it also depends on your use of fakes as described in section 3.5.2).

But that's the procedure for creating new native tests:

1) Add the necessary .c and .h files in the src/ folder
2) Update the `test_list.mk` to specify how you want your tests structured across one or more flashable files.

For native tests there's an extra option for defining tests because cmock is available. It looks like this:

$(eval $(call TEST_DEFINITION, test_mul_op.c, add_op.c sub_op.c, mul_op.h))

Test file — "Production" .c files — "Production" .h files

There's an extra "3rd class" of files: .h files from the production code. Made3DevOps will automatically use cmock to create fakes for all the functions found in the .h files listed in the `test_list.mk` file in the native_test folder. If you don't want to use cmock, just leave the section with .h files empty (but the extra comma has to be in there). Look at the existing examples to see how it's done.

3.6 Create and run new firmware and tests on Arduino

Once you have added or changed any files that goes into code that should run on the Arduino, simply choose the relevant task from the task menu to build the flashable file(s).
You can choose to build all target code in two different configurations, release or debug. Depending on which you choose, the code will be compiled with a predefined constants:

- Release config defines these constants — #define RELEASE
#define NDEBUG
- Debug config defines these constants — #define DEBUG

You use this in your code like this:

```
#ifdef DEBUG
// Any put code here will be disabled when the software is built in release config
#endif
```

Once you have built the code, you can choose another task that flashes the code to the Arduino. Alternatively, if you request code to be flashed the relevant subproject will automatically be rebuilt to make sure that the flashed code is fully up to date.

3.7 Create and run new native/host tests

Once you have added or changed any files that goes into code that should run on your laptop, simply choose the relevant task from the task menu to build and run the relevant tests. Like target code, native code can be built in release and debug configurations.

If you build and run your native test code in debug config, a test coverage report is automatically generated. It will be placed in the folder build/native_test_/debug/testreport. If you right-click the file called index.html like this:

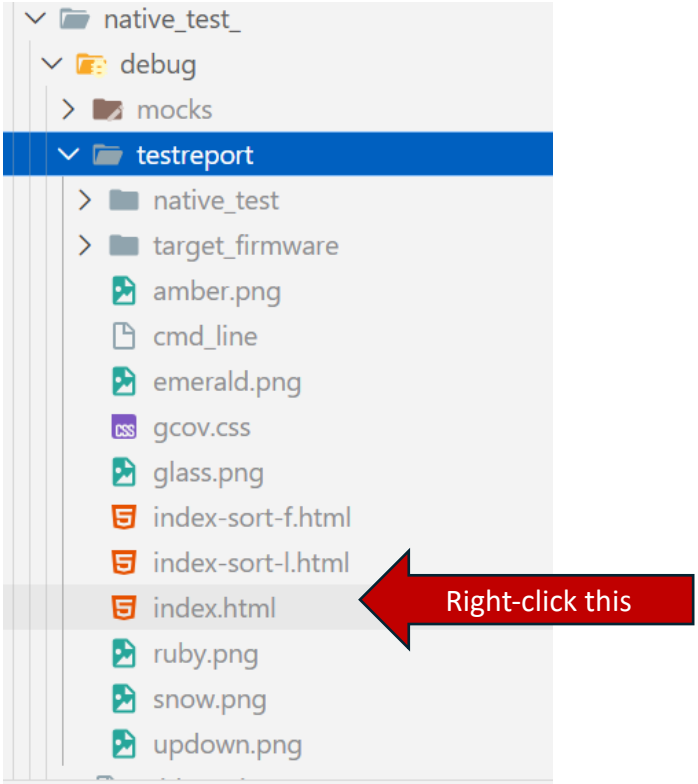


FIGURE 6 - VIEWING TEST COVERAGE REPORT

You will then see a nice test coverage report in your browser.

3.8 Push updated project to GitHub

The Made4DevOps "master project" https://github.com/DaFessor/made4devops has been set up with good support for CI on Github:

1) The main branch is protected so that it is impossible to push updates directly into the branch on Github
2) Use of pull requests to get stuff onto main is forced
3) There's a Github workflow that performs a check on incoming changes
    a. All code can compile (target firmware, target tests, native tests)
    b. All native tests run without errors

If anything fails in step 3 the pull request fails, and you must update it before the changes can be merged onto main.

3.9 View info on latest version on GitHub

The CI workflow mentioned above also does some extra things if the pull request is approved:

4) It publishes the latest version of the target firmware (a .hex file, ready to be flashed) on a Github pages website associated with the project
5) It publishes the test coverage for native tests for the latest version

You can see the result here: https://dafessor.github.io/made4devops/ . So it's easy to always get the latest version of the Arduino firmware, ready to flash, together with a matching test coverage report.

3.10 Stop a session

To stop a session, do two things:

1) Choose “Close Remote Connection” from the File menu in VSCode
2) Press Ctrl-C in the Powershell window where usbipd is running (or close the Powershell window where usbipd is running)

4 Removing unnecessary software

If you don’t want to use Made4DevOps anymore, you can of course delete the software packages involved. But since you probably want to keep VSCode, Docker Desktop and WSL2, in most cases you just need to uninstall usbipd. That’s easy, just go to “Settings/Apps/Installed apps” and find *usbipd-win* and uninstall it.

5 Extra features

5.1 Sep4 hardware library

Made4devop comes full support for the sep4 driver library. The library is automatically installed as an external dependency when you build software for the target (Arduino). The dependency manager automatically fetches the source code for the driver library and compiles it into a linkable library that’s added to all target builds.

5.2 Sep4 HW test

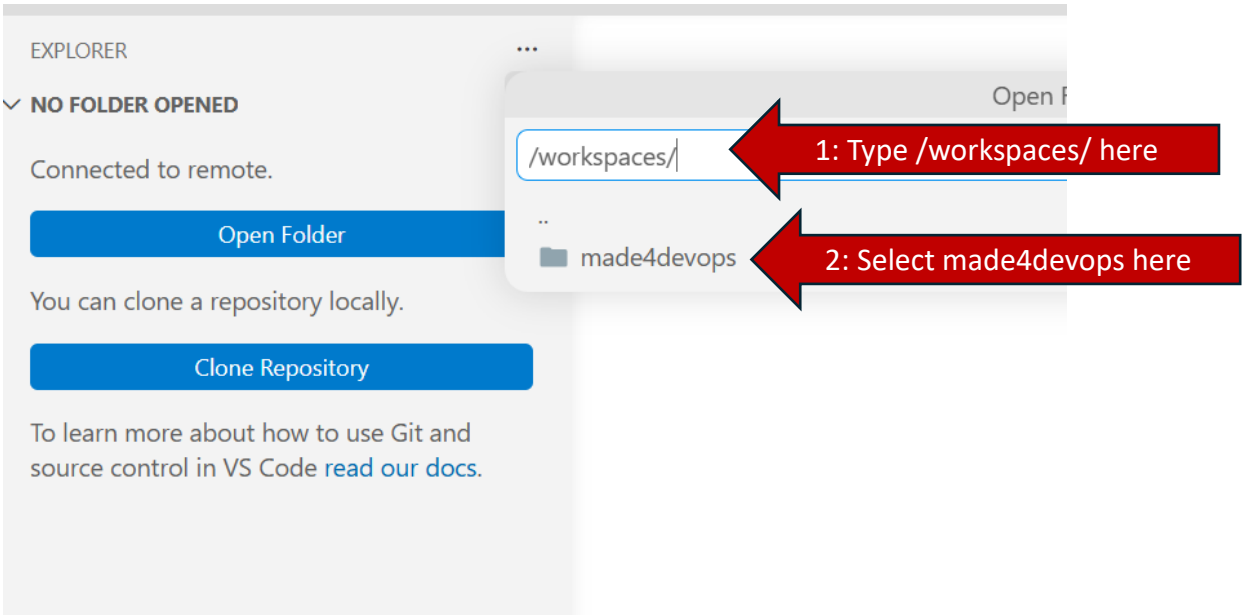
Once you have the project loaded in VSCode, you can check-out the branch called `sep4_hw_test`. In this branch the code in the target_firmware subproject contains the code for the sep4 hardware test. So, after checking out this branch, all you need to do to run the hw-test is choose “Run target fw (release)” from the task menu.

6 How do I ...

6.1 - get access to the raw project folder?

Sometimes you may want to see or work with the “raw” contents of the checked-out project folder, rather than the simplified virtual view the created by opening the .code-workspace file. To do so you just need to:

1. Choose “Close Workspace” from the VSCode File menu. This closes the virtual workspace and restores the raw, untranslated view of the project folder.
2. But first you have to choose a folder when VSCode prompts you to open a folder inside the devcontainer, do like this:



3. Do your stuff.
4. Choose “Open Workspace from File...” from the VSCode File menu and (re-) open the .code-workspace file when you want to go back to the simplified/prettified virtual view.

6.2 - reconnect to an Arduino that’s already running my software?

If your Arduino is already running the software you want it to run, and you just want to reconnect to it (i.e. you don’t need or want to flash the software again), you can use the task “Reconnect to target” in the Root project part of the task menu. This will reestablish the connection to the Arduino. But be aware that the Arduino always gets reset when a connection is established. This is a feature that’s baked into the Arduino (it can be disabled so that only a press on the reset button causes a reset, but it requires modifying the Arduino hardware)

7 Work flow overview

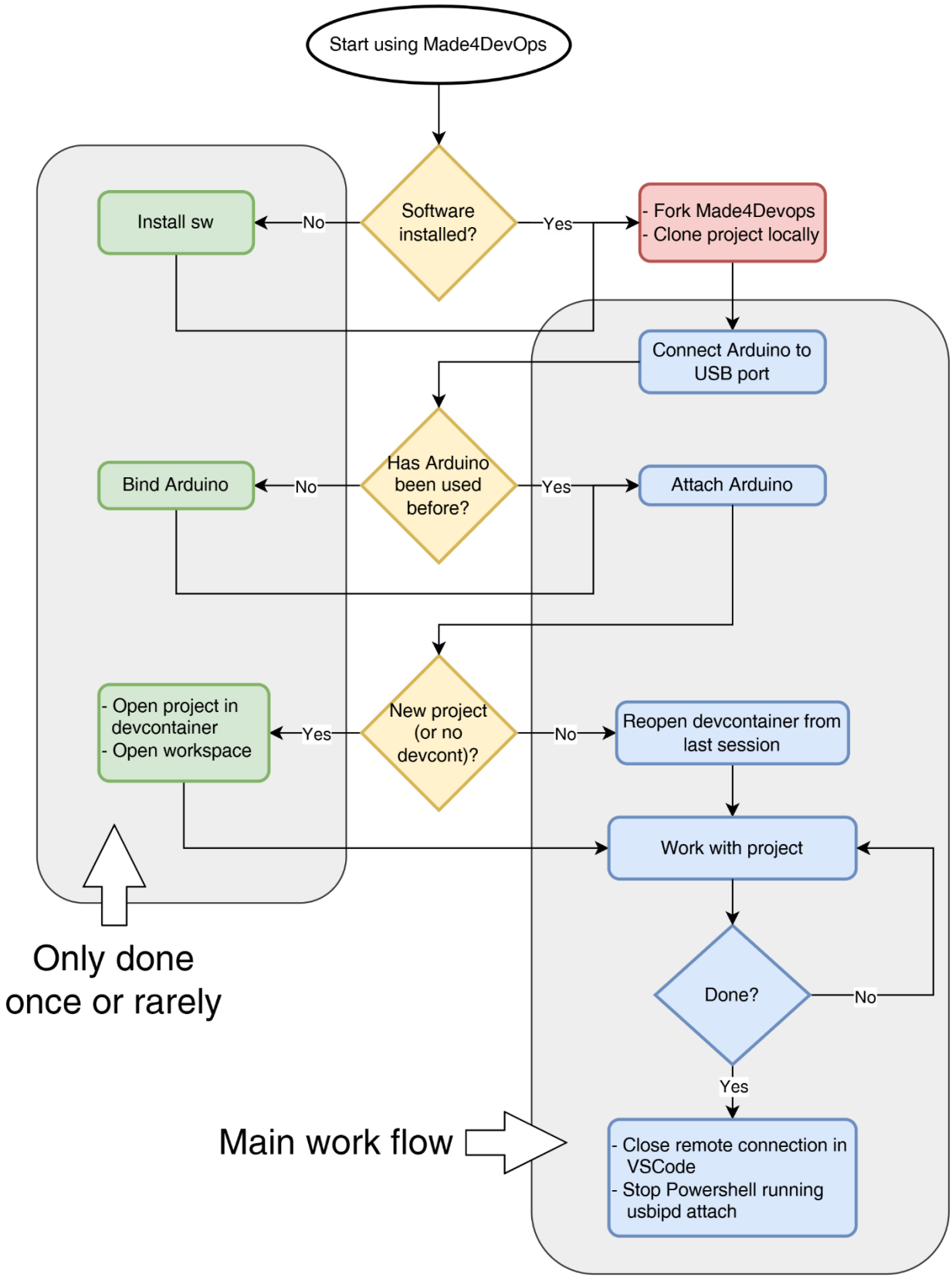


FIGURE 7 - WORKFLOW OVERVIEW